NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1

Aim: Study and Implementation of Basic Git and GitHub Operations.

This Experiment maps to LO2.

LO2: To understand and perform the basic version control operations using Git and GitHub, including repository creation, cloning, file tracking, committing, pushing, pulling, branching, merging, and remote repository management.

Theory:

Git is a distributed version control system that helps developers track changes in source code and files. It enables efficient project management through repositories, commits, branches, and remote repositories, making collaboration and version management easier. In this experiment, various Git commands are performed to understand the basic operations of version control.

Commands:

git init : Initializes a new Git repository in the current working directory.

git clone <repository-url> : Creates a local copy of an existing remote GitHub repository.

git status : Displays the current state of the repository, including staged, modified, and untracked files.

git add <file> : Adds a specific file to the staging area before committing.

git add . : Stages all newly created and modified files in the current directory.

git commit -m "message" : Records the staged changes in the local repository with a descriptive commit message.

git log : Shows the complete history of commits made in the repository.

git log --oneline : Displays a concise, one-line summary of each commit.

git diff : Shows the differences between the working directory and the most recent commit.

git branch : Lists all available local branches and indicates the active branch.

git branch <branch-name> : Creates a new branch without switching to it.

git checkout <branch-name> : Switches from the current branch to the specified branch.

NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1

git checkout -b <branch-name> : Creates a new branch and immediately switches to it.

git switch <branch-name> : Switches to an existing branch using the modern Git command.

git merge <branch-name> : Merges the specified branch into the currently active branch.

git remote -v : Displays the remote repositories linked to the local repository.

git remote add origin <repository-url> : Connects the local repository to a remote GitHub repository.

git push origin <branch-name> : Uploads local commits from the specified branch to the remote repository.

git pull origin <branch-name> : Fetches and merges the latest changes from the remote repository into the current branch.

git fetch : Downloads updates from the remote repository without merging them into the current branch.

git reset --hard HEAD : Removes all uncommitted changes and restores the working directory to the latest commit.

git stash : Temporarily saves uncommitted changes without creating a commit.

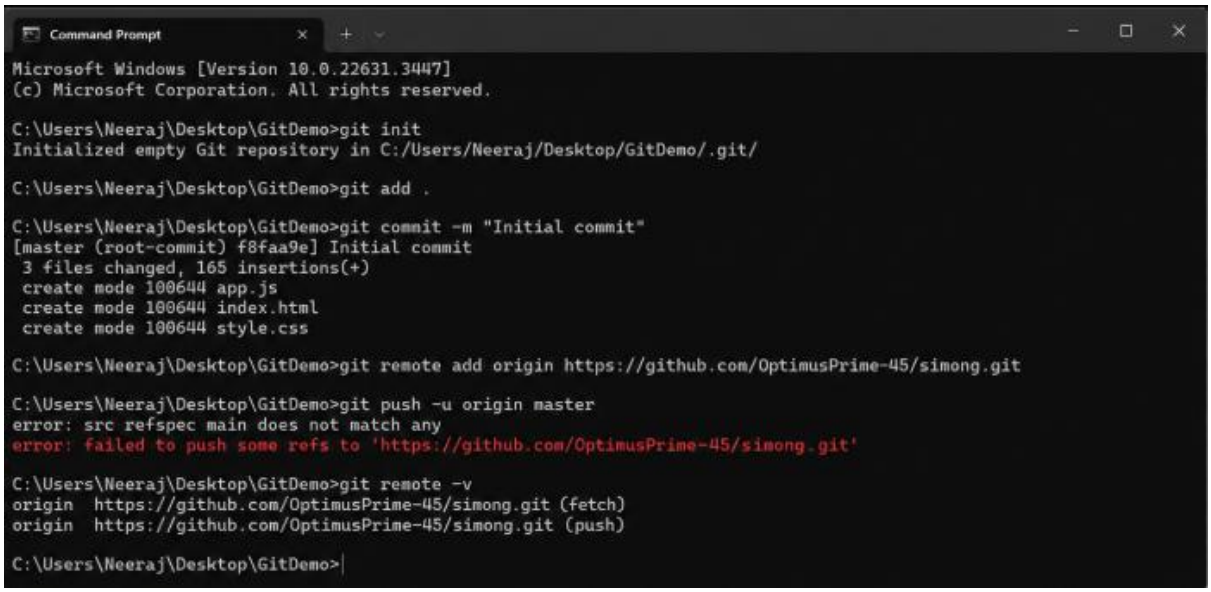
git stash pop : Restores the most recently stashed changes and removes them from the stash list.

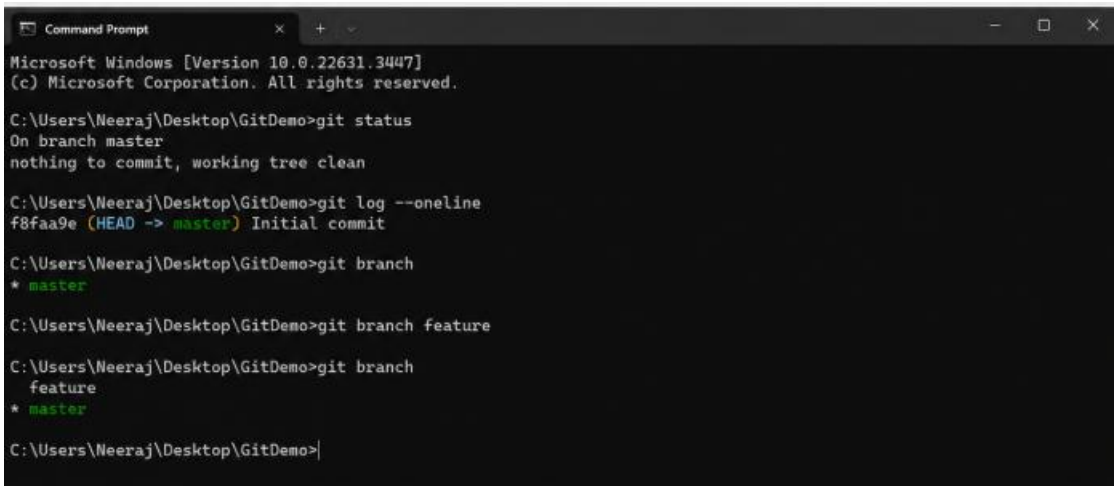
git rm <file> : Deletes a file from both the working directory and the Git repository.

git config --global user.name "Your Name" : Sets the global username that Git uses for commits.

NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

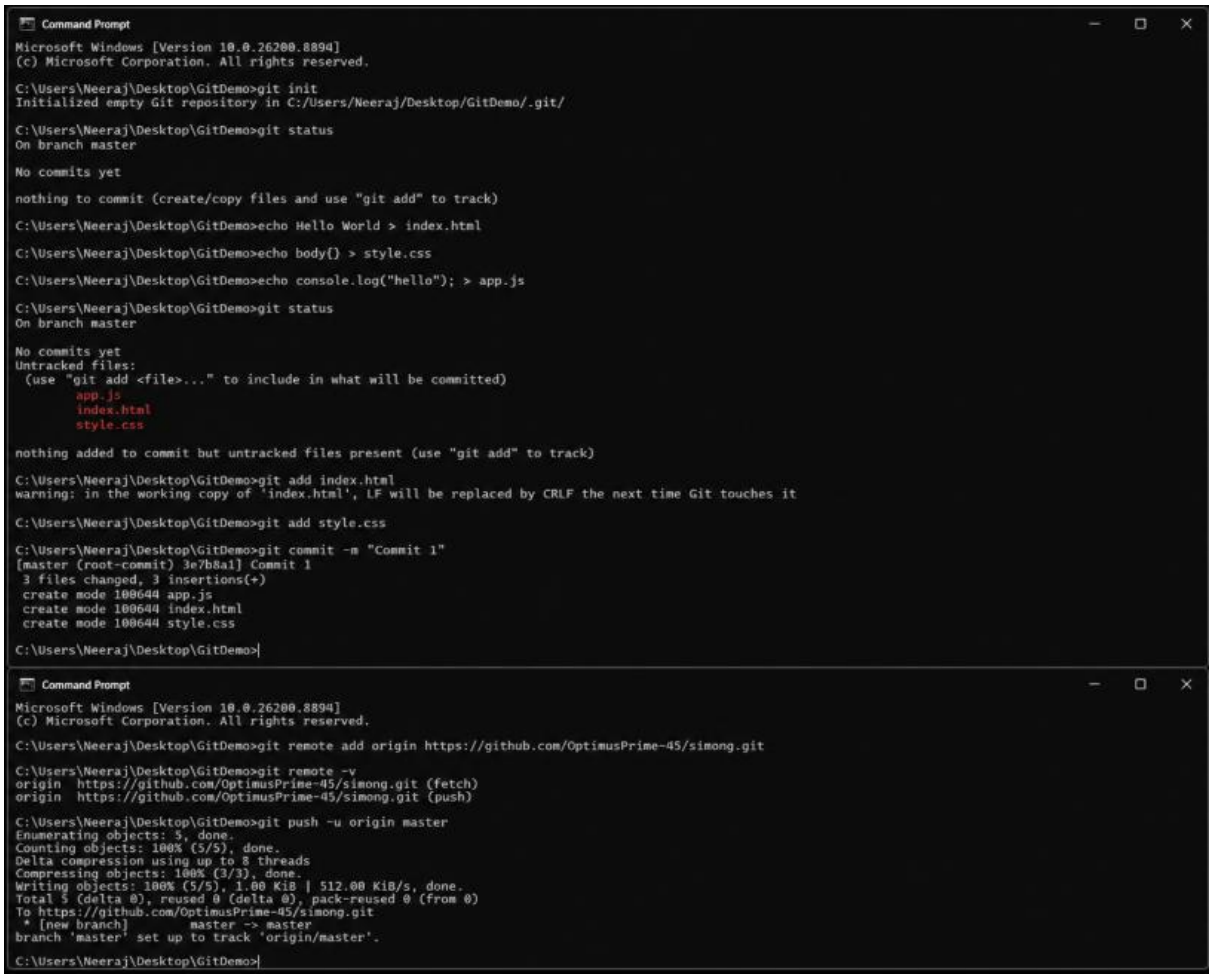
EXPERIMENT – 1





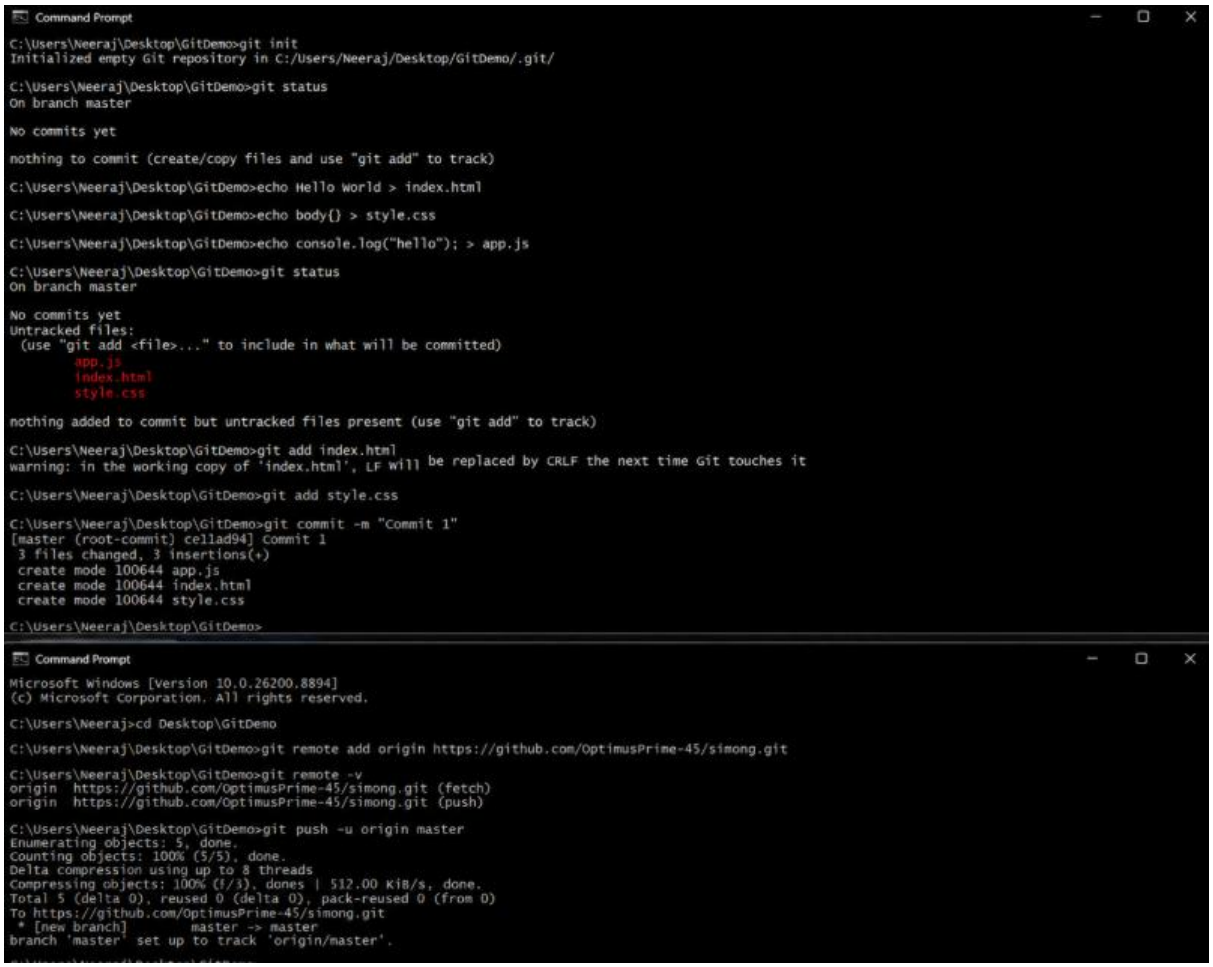
NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1



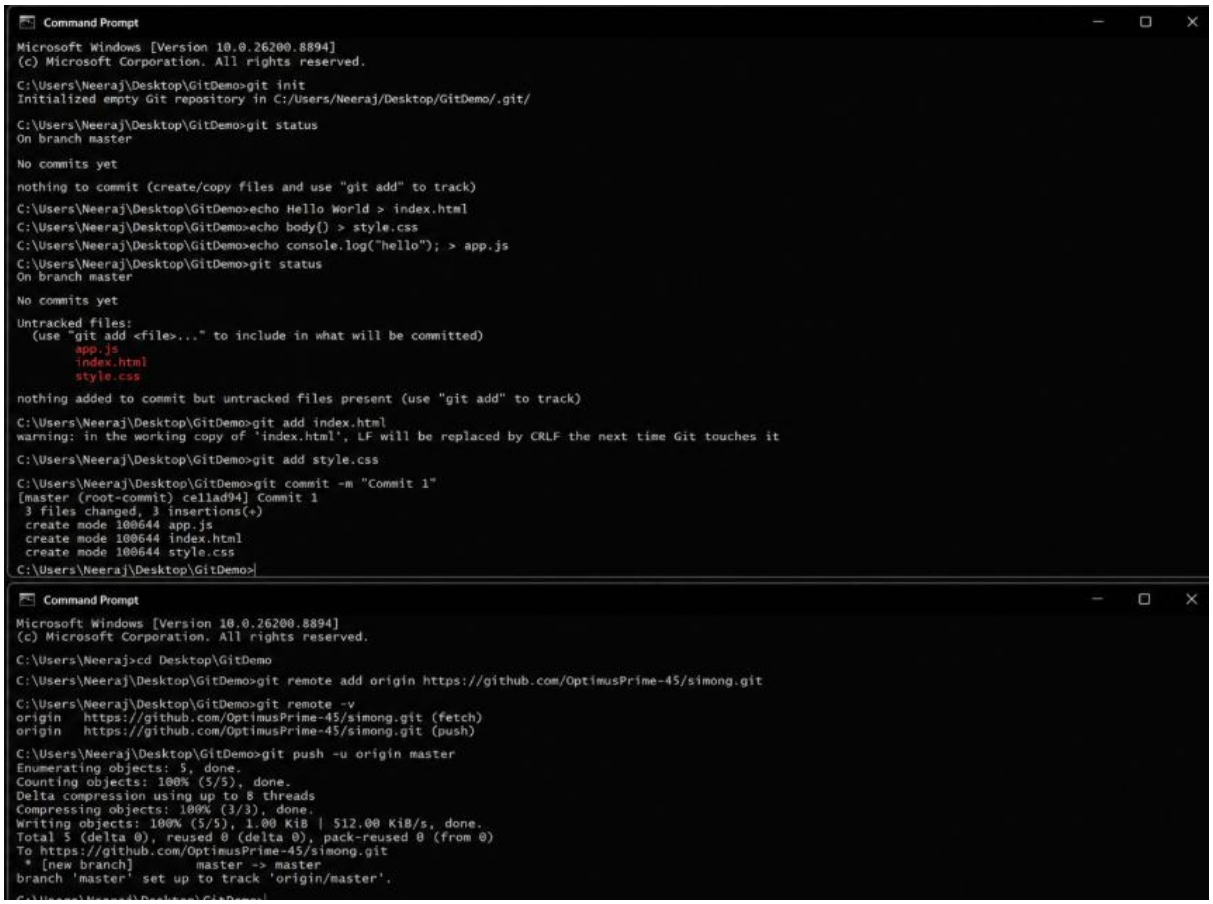
NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1



NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1



NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1



NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1

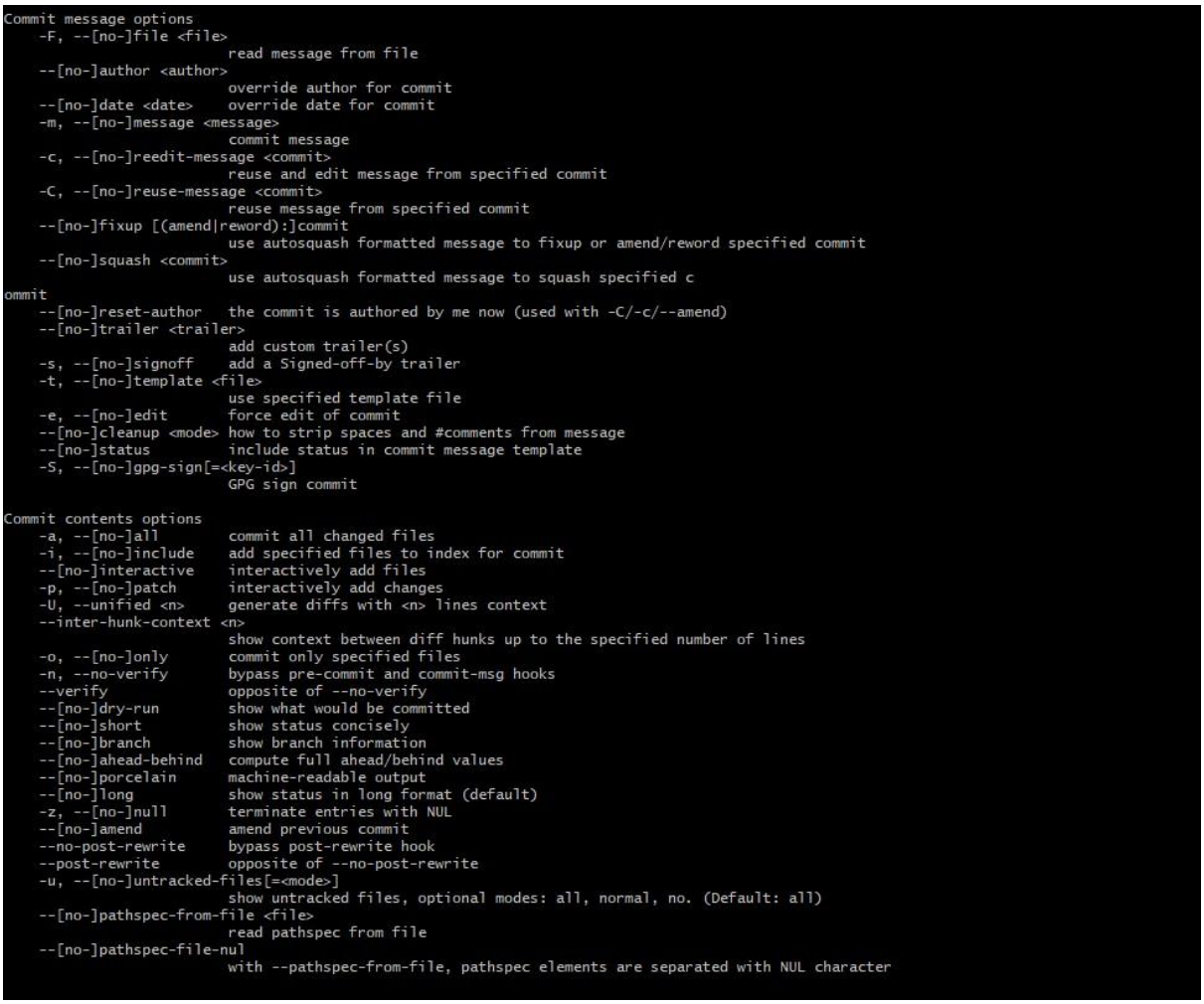
```
    -q, --[no-]quiet      suppress summary after successful commit
    -v, --[no-]verbose    show diff in commit message template

Commit message options
    -F, --[no-]file <file>
                          read message from file
    --[no-]author <author>
                          override author for commit
    --[no-]date <date>    override date for commit
    -m, --[no-]message <message>
                          commit message
    -c, --[no-]reedit-message <commit>
                          reuse and edit message from specified commit
    -C, --[no-]reuse-message <commit>
                          reuse message from specified commit
    --[no-]fixup [(amend|reword):]commit
                          use autosquash formatted message to fixup or amend/reword specified commit
    --[no-]squash <commit>
                          use autosquash formatted message to squash specified c
ommit
    --[no-]reset-author   the commit is authored by me now (used with -C/-c/--amend)
    --[no-]trailer <trailer>
                          add custom trailer(s)
    -s, --[no-]signoff    add a Signed-off-by trailer
    -t, --[no-]template <file>
                          use specified template file
    -e, --[no-]edit       force edit of commit
    --[no-]cleanup <mode> how to strip spaces and #comments from message
    --[no-]status         include status in commit message template
    -S, --[no-]gpg-sign[=<key-id>]
                          GPG sign commit

Commit contents options
    -a, --[no-]all        commit all changed files
    -i, --[no-]include    add specified files to index for commit
    --[no-]interactive    interactively add files
    -p, --[no-]patch      interactively add changes
    -U, --unified <n>     generate diffs with <n> lines context
    --inter-hunk-context <n>
                          show context between diff hunks up to the specified number of lines
    -o, --[no-]only       commit only specified files
    -n, --no-verify       bypass pre-commit and commit-msg hooks
    --verify              opposite of --no-verify
    --[no-]dry-run        show what would be committed
    --[no-]short          show status concisely
    --[no-]branch         show branch information
    --[no-]ahead-behind   compute full ahead/behind values
    --[no-]porcelain      machine-readable output
    --[no-]long           show status in long format (default)
    -z, --[no-]null       terminate entries with NUL
    --[no-]amend          amend previous commit
    --no-post-rewrite     bypass post-rewrite hook
    --post-rewrite        opposite of --no-post-rewrite
```

NAME: Neeraj Pandey
ROLL NO.: 2401073
BATCH: T21
BRANCH: AI&DS

EXPERIMENT – 1



```
Commit message options
    -F, --[no-]file <file>
                          read message from file
    --[no-]author <author>
                          override author for commit
    --[no-]date <date>    override date for commit
    -m, --[no-]message <message>
                          commit message
    -c, --[no-]reedit-message <commit>
                          reuse and edit message from specified commit
    -C, --[no-]reuse-message <commit>
                          reuse message from specified commit
    --[no-]fixup [(amend|reword):]commit
                          use autosquash formatted message to fixup or amend/reword specified commit
    --[no-]squash <commit>
                          use autosquash formatted message to squash specified c
ommit
    --[no-]reset-author   the commit is authored by me now (used with -C/-c/--amend)
    --[no-]trailer <trailer>
                          add custom trailer(s)
    -s, --[no-]signoff    add a Signed-off-by trailer
    -t, --[no-]template <file>
                          use specified template file
    -e, --[no-]edit       force edit of commit
    --[no-]cleanup <mode> how to strip spaces and #comments from message
    --[no-]status         include status in commit message template
    -S, --[no-]gpg-sign[=<key-id>]
                          GPG sign commit

Commit contents options
    -a, --[no-]all        commit all changed files
    -i, --[no-]include    add specified files to index for commit
    --[no-]interactive    interactively add files
    -p, --[no-]patch      interactively add changes
    -U, --unified <n>     generate diffs with <n> lines context
    --inter-hunk-context <n>
                          show context between diff hunks up to the specified number of lines
    -o, --[no-]only       commit only specified files
    -n, --no-verify       bypass pre-commit and commit-msg hooks
    --verify              opposite of --no-verify
    --[no-]dry-run        show what would be committed
    --[no-]short          show status concisely
    --[no-]branch         show branch information
    --[no-]ahead-behind   compute full ahead/behind values
    --[no-]porcelain      machine-readable output
    --[no-]long           show status in long format (default)
    -z, --[no-]null       terminate entries with NUL
    --[no-]amend          amend previous commit
    --no-post-rewrite     bypass post-rewrite hook
    --post-rewrite        opposite of --no-post-rewrite
    -u, --[no-]untracked-files[=<mode>]
                          show untracked files, optional modes: all, normal, no. (Default: all)
    --[no-]pathspec-from-file <file>
                          read pathspec from file
    --[no-]pathspec-file-nul
                          with --pathspec-from-file, pathspec elements are separated with NUL character
```

CONCLUSION

The experiment demonstrated the use of essential Git version control commands for repository management, tracking changes, branch handling, and collaboration with GitHub. It provided practical knowledge of managing and maintaining software projects efficiently using Git.